

Комплексное задание: «Каталог строительных элементов»

Сквозной учебный проект по главам 3–7 пособия *Java from EPAM*. Один домен растёт от задания к заданию: каждое задание = одна тема + несколько пунктов + один дизайн-паттерн. Доходишь до конца — получаешь работающее мини-приложение, написанное на знаниях глав 1–7 (без коллекций, Stream API, исключений «по-взрослому» — этого ещё не было).

Домен. Каталог элементов здания: стены, перекрытия, окна и т.п. У каждого элемента есть параметры (имя, материал, габариты) — концептуально похоже на элементы модели в BIM, но это чистая Java.

Правило хранения коллекций. Главу «Коллекции» ты ещё не проходил, поэтому везде используем **обычные массивы** (`BuildingElement[]`). Это намеренное ограничение — оно заставит поработать с массивами из главы 2.

Задание 1. Классы и методы (глава 3)

Тема: инкапсуляция, конструкторы, `enum`, `record`, `Optional`, `varargs`, параметризованный метод.

Паттерн: `Singleton` (реестр единиц измерения).

Пункты:

1. Создай `enum Material { CONCRETE, BRICK, STEEL, GLASS, WOOD }` с полем «плотность, кг/м³» и геттером. Плотность задаётся в конструкторе `enum`.
2. Создай **record** `Dimensions(double length, double width, double height)` с методом `double volume()`. Проверь в компактном конструкторе, что все размеры > 0 (пока без своих исключений — кидай стандартный `IllegalArgumentException`).
3. Создай класс `BuildingElement` с приватными полями: `name`, `Material material`, `Dimensions dimensions`. Полная инкапсуляция, конструктор, геттеры.
4. Добавь метод `double mass()` — масса элемента через объём и плотность материала.
5. Реализуй `Optional<String>`
`findName(BuildingElement[] elements, int index)` — возвращает имя по индексу или `Optional.empty()`, если индекс вне массива. Никаких `null` наружу.
6. Напиши **параметризованный** статический метод `<T> T firstOrNull(T[] array)` и метод с `varargs` `static double totalMass(BuildingElement... elements)`.
7. **Singleton** `UnitRegistry` (приватный конструктор, статический `getInstance()`) с методом `String format(double mass) → "1234.5 kg"`. Везде, где выводишь массу, форматируй через него.

Проверка: в `main` создай 3–4 элемента, посчитай суммарную массу через `varargs`, выведи через `Singleton`.

Задание 2. Наследование и полиморфизм (глава 4)

Тема: иерархия, `abstract`, переопределение методов, `equals` / `hashCode` / `toString` (класс `Object`), `clone`, восходящее преобразование.

Паттерн: `Template Method`.

Пункты:

1. Сделай `BuildingElement` **абстрактным** базовым классом. Добавь абстрактный метод `String category()`.
2. Наследники: `Wall`, `Slab` (перекрытие), `Window`. У `Window` добавь поле `boolean openable`, у `Wall` — `boolean loadBearing` (несущая).
3. Переопредели `equals()`, `hashCode()`, `toString()` в `BuildingElement` (имя + материал + габариты). У наследников расширь `toString()` через `super.toString()`.
4. Реализуй интерфейс `Cloneable` и метод `clone()` так, чтобы `Dimensions`-копия была независимой (но `record` неизменяем — обсуди в комментариях, нужно ли глубокое копирование и почему здесь нет).
5. Полиморфизм: метод `static void printCatalog(BuildingElement[] elements)` — печатает `category()` и `toString()` для каждого, не зная конкретного типа.
6. **Template Method:** в базовом классе сделай `final String report()`, который вызывает `header()` (общий) → `details()` (абстрактный, реализуют

наследники) → `footer()` (общий). Шаблон фиксирован в базе, шаги переопределяются.

Проверка: массив из разных наследников, прогон через `printCatalog` и `report()`; проверь `equals` на двух одинаковых стенах.

Задание 3. Внутренние классы (глава 5)

Тема: вложенный (`static nested`), внутренний (`inner`), анонимный класс.

Паттерн: **Builder** (на `static nested` классе) + **Iterator** (на `inner` классе).

Пункты:

1. Создай класс `Building` (здание), который внутри хранит `BuildingElement[]` и `String address`.
2. **Builder (`static nested`):** `Building.Builder` со методами `withAddress(...)`, `addElement(...)` (по цепочке, возвращают `this`) и `build()`. Конструктор `Building` сделай приватным — собирать здание можно только через `Builder`.
3. **Iterator (`inner`):** напиши внутренний (нестатический) класс `ElementIterator` с методами `boolean hasNext()` / `BuildingElement next()`, который обходит массив элементов здания. Внутренний класс — потому что ему нужен доступ к полям внешнего объекта.

4. Добавь в `Building` метод `ElementIterator iterator()`.
5. **Анонимный класс:** метод `BuildingElement[] filter(...)`, принимающий анонимную реализацию интерфейса `ElementCondition { boolean test(BuildingElement e); }`. В `main` передай анонимный класс «только несущие стены».

Проверка: собери здание билдером, пройди итератором в цикле `while`, отфильтруй элементы анонимным классом.

Замечание: пункт 5 — это «ручной» Strategy через анонимный класс. В задании 5 ты перепишешь его на лямбды и почувствуешь разницу.

Задание 4. Интерфейсы и аннотации (глава 6)

Тема: интерфейсы, параметризация интерфейсов, своя аннотация + обработка через рефлексию.

Паттерн: Strategy + Factory Method.

Пункты:

1. Параметризованный интерфейс `Repository<T>` с методами `void add(T item)`, `T get(int i)`, `int size()`, `T[] toArray()`.
2. Реализация `ArrayRepository<T>` поверх массива (с ручным расширением — `Arrays.copyOf` уже можно).

3. **Strategy:** интерфейс `PriceStrategy { double price(BuildingElement e); }` и две реализации: `ByMassPrice` (цена за кг) и `ByVolumePrice` (цена за м³). Класс `Estimator` принимает стратегию в конструкторе и считает смету по массиву.
4. **Factory Method:** интерфейс `ElementFactory { BuildingElement create(String name, Material m, Dimensions d); }` и фабрики `WallFactory`, `WindowFactory`. Клиентский код работает с `ElementFactory`, не зная конкретного класса.
5. **Своя аннотация** `@Component(name = "...")` с `@Retention(RUNTIME)` и `@Target(TYPE)`. Помечай ею классы-фабрики.
6. Напиши `static void describe(Object o)` — через рефлексию проверяет наличие `@Component` и печатает `name`, иначе сообщает, что аннотации нет.

Проверка: собери смету двумя стратегиями на одном массиве, сравни; создай элементы через фабрики; прогони фабрики через `describe`.

Задание 5. Функциональные интерфейсы (глава 7)

Тема: `Predicate`, `Function`, `Consumer`, `Supplier`, `Comparator`, `default / static` методы интерфейса, замыкания, ссылки на методы.

Паттерн: **Command + Observer** (оба на функциональных интерфейсах) + **Strategy** на лямбдах.

Пункты:

1. **Predicate:** перепиши `filter` из задания 3 на `Predicate<BuildingElement>`. Скомбинируй условия через `.and()` / `.or()` / `.negate()` (стеклянные ИЛИ несущие).
2. **Function:** `Function<BuildingElement, Double> toMass = BuildingElement::mass;` — используй **ссылку на метод**. Скомбинируй через `.andThen(...)` (масса → строка через `UnitRegistry`).
3. **Comparator:** отсортируй `BuildingElement[]` по массе через `Comparator.comparingDouble(...)`, потом по имени через `.thenComparing(...)`, и в обратном порядке через `.reversed()`.
4. **Supplier как Factory:** перепиши фабрики из задания 4 на `Supplier<BuildingElement>` / лямбды — почувствуй, что отдельный интерфейс-фабрика часто не нужен.
5. **Strategy на лямбдах:** перепиши `PriceStrategy` из задания 4 как лямбды (`e -> e.mass() * 5.0`). Сравни с реализацией через классы — это та же стратегия, но компактнее.
6. **Command:** интерфейс `Command { void execute(); }` (по сути `Runnable`) — массив команд «добавить элемент», «пересчитать смету», «вывести каталог». Выполни их в цикле. Покажи **замыкание**: команда захватывает локальную переменную из метода.
7. **Observer на Consumer:** в `Building` добавь `Consumer<BuildingElement> onElementAdded;` при добавлении элемента уведомляй слушателя (например, печать в лог). Подпишись лямбдой.

8. **default + static в интерфейсе:** в `PriceStrategy`

добавь `default PriceStrategy withMarkup(double percent)` (возвращает новую стратегию с наценкой — снова замыкание) и `static PriceStrategy free()`.

Проверка: один массив элементов, прогон через все механизмы — фильтр, сортировка, смета со стратегией + наценкой, паттерн Observer при добавлении.

Финальная интеграция

Собери всё в одном `main`:

1. Билдером (зад. 3) создай `Building` с подпиской-Observer (зад. 5).
 2. Элементы создавай через `Supplier`-фабрики (зад. 5).
 3. Отфильтруй несущие стены `Predicate` (зад. 5), отсортируй по массе `Comparator`.
 4. Посчитай смету `Estimator` со Strategy + `withMarkup(10)` (зад. 4–5).
 5. Выведи `report()` каждого элемента (Template Method, зад. 2).
 6. Всё форматирование массы — через Singleton `UnitRegistry` (зад. 1).
-

Карта паттернов

Паттерн	Где	На чём построен
Singleton	<code>UnitRegistry</code>	static + private конструктор
Template Method	<code>BuildingElement.report()</code>	abstract + final
Builder	<code>Building.Builder</code>	static nested класс
Iterator	<code>ElementIterator</code>	inner класс
Factory Method	<code>ElementFactory</code>	интерфейс → Supplier
Strategy	<code>PriceStrategy</code>	интерфейс → лямбда
Command	<code>Command</code> / <code>Runnable</code>	функциональный интерфейс
Observer	<code>onElementAdded</code>	<code>Consumer<T></code>

Критерии готовности

- Нет утечки `null` наружу — где может не быть значения, там `Optional`.
- Каждый класс — одна ответственность; поля приватные.
- `equals` / `hashCode` / `toString` согласованы.
- Один и тот же паттерн (Strategy, Factory) реализован двумя способами: через классы (зад. 4) и через

лямбды (зад. 5) — это и есть главный вывод глав 6–7.

- Код по Java Code Conventions: имена, отступы, javadoc на публичных методах.

Подсказки к подводным камням

- `record` неизменяем — `clone()` для `Dimensions` не нужен; объясни это в комментарии (зад. 2, п. 4).
- В компактном конструкторе `record` нельзя присваивать `this.field` до неявного присвоения — просто проверяй и кидай исключение.
- Внутренний (`inner`) класс держит ссылку на внешний объект, `static nested` — нет. `Builder` делай именно `static nested`.
- `Comparator.comparingDouble` берёт `ToDoubleFunction` — туда отлично ложится ссылка на метод `BuildingElement::mass`.